

```
[11]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
[12]: df = pd.read_csv('website_data.csv')
```

```
[13]: df.head()
```

# ----- ----- ----- ----										
		Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9
0	Session primary channel group (Default channel...	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count
1	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088610	4.673333333333330	0.48	1402
2	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153850	4.295880149812730	0.4943820224719100	1147
3	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595740	4.587982832618030	0.49356223175965700	1069
4	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609630	4.078125	0.48828125	1044

```
[14]: df.columns = df.iloc[0]
df.head()
```



[14]:

	Session primary channel group (Default channel group)	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count
0	Session primary channel group (Default channel...	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count
1	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088610	4.673333333333330	0.48	1402
2	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153850	4.295880149812730	0.4943820224719100	1147
3	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595740	4.587982832618030	0.49356223175965700	1069
4	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609630	4.078125	0.48828125	1044

```
[29]: df = df.drop(index=0).reset_index(drop=True)
df.columns = ["channel group", "Date Hour", "Users", "Sessions", "Engaged Sessions", "Average engagement time per session", "Engaged sessions per user",
```

[30]:	df.head()										
[30]:		channel group	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count
	0	Organic Social	2024041721	160	206	103	59.31553398058250	0.64375	4.694174757281550	0.5	967
	1	Organic Social	2024050119	160	219	110	40.31506849315070	0.6875	4.63013698630137	0.502283105022831	1014
	2	Direct	2024041622	158	192	94	39.020833333333300	0.5949367088607590	4.057291666666670	0.4895833333333330	779
	3	Organic Social	2024041321	152	178	96	30.48876404494380	0.631578947368421	5.056179775280900	0.5393258426966290	900
	4	Organic Social	2024041516	151	211	93	64.16587677725120	0.6158940397350990	4.317535545023700	0.44075829383886300	911

```
[31]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 3177 entries, 0 to 3176
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   channel group                        3177 non-null   str
1   Date Hour                           3177 non-null   str
2   Users                               3177 non-null   str
3   Sessions                            3177 non-null   str
4   Engaged Sessions                    3177 non-null   str
5   Average engagement time per session 3177 non-null   str
6   Engaged sessions per user           3177 non-null   str
7   Events per session                  3177 non-null   str
8   Engagement rate                     3177 non-null   str
9   Event count                         3177 non-null   str
dtypes: str(10)
memory usage: 248.3 KB
```

```
[33]: df["Date Hour"] = pd.to_datetime(df["Date Hour"], format = '%Y%m%d%H', errors = 'coerce')
df.head()
```

[33]:	channel group	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count
0	Organic Social	2024-04-17 21:00:00	160	206	103	59.31553398058250	0.64375	4.694174757281550	0.5	967
1	Organic Social	2024-05-01 19:00:00	160	219	110	40.31506849315070	0.6875	4.63013698630137	0.502283105022831	1014
2	Direct	2024-04-16 22:00:00	158	192	94	39.0208333333333300	0.5949367088607590	4.0572916666666670	0.48958333333333330	779

3	Organic Social	2024-04-13 21:00:00	152	178	96	30.48876404494380	0.631578947368421	5.056179775280900	0.5393258426966290	900
4	Organic Social	2024-04-15 16:00:00	151	211	93	64.16587677725120	0.6158940397350990	4.317535545023700	0.44075829383886300	911

4]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3177 entries, 0 to 3176
Data columns (total 10 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   channel group                        3177 non-null   str
 1   Date Hour                           3177 non-null   datetime64[us]
 2   Users                               3177 non-null   str
 3   Sessions                            3177 non-null   str
 4   Engaged Sessions                    3177 non-null   str
 5   Average engagement time per session 3177 non-null   str
 6   Engaged sessions per user           3177 non-null   str
 7   Events per session                  3177 non-null   str
 8   Engagement rate                     3177 non-null   str
 9   Event count                         3177 non-null   str
dtypes: datetime64[us](1), str(9)
memory usage: 248.3 KB
```

5]:

```
numeric_cols = df.columns.drop(["channel group", "Date Hour"])
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric,errors='coerce')
df['Hour']=df['Date Hour'].dt.hour
```


7]:	df.head()										
7]:	channel group	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count	Hour
0	Organic Social	2024-04-17 21:00:00	160	206	103	59.315534	0.643750	4.694175	0.500000	967	21
1	Organic Social	2024-05-01 19:00:00	160	219	110	40.315068	0.687500	4.630137	0.502283	1014	19
2	Direct	2024-04-16 22:00:00	158	192	94	39.020833	0.594937	4.057292	0.489583	779	22
3	Organic Social	2024-04-13 21:00:00	152	178	96	30.488764	0.631579	5.056180	0.539326	900	21
4	Organic Social	2024-04-15 16:00:00	151	211	93	64.165877	0.615894	4.317536	0.440758	911	16

```
[38]: df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 3177 entries, 0 to 3176
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	channel group	3177 non-null	str
1	Date Hour	3177 non-null	datetime64[us]
2	Users	3177 non-null	int64
3	Sessions	3177 non-null	int64
4	Engaged Sessions	3177 non-null	int64
5	Average engagement time per session	3177 non-null	float64
6	Engaged sessions per user	3177 non-null	float64
7	Events per session	3177 non-null	float64
8	Engagement rate	3177 non-null	float64
9	Event count	3177 non-null	int64
10	Hour	3177 non-null	int32

```
dtypes: datetime64[us](1), float64(4), int32(1), int64(4), str(1)
```

```
memory usage: 260.7 KB
```

[39]: df.describe()

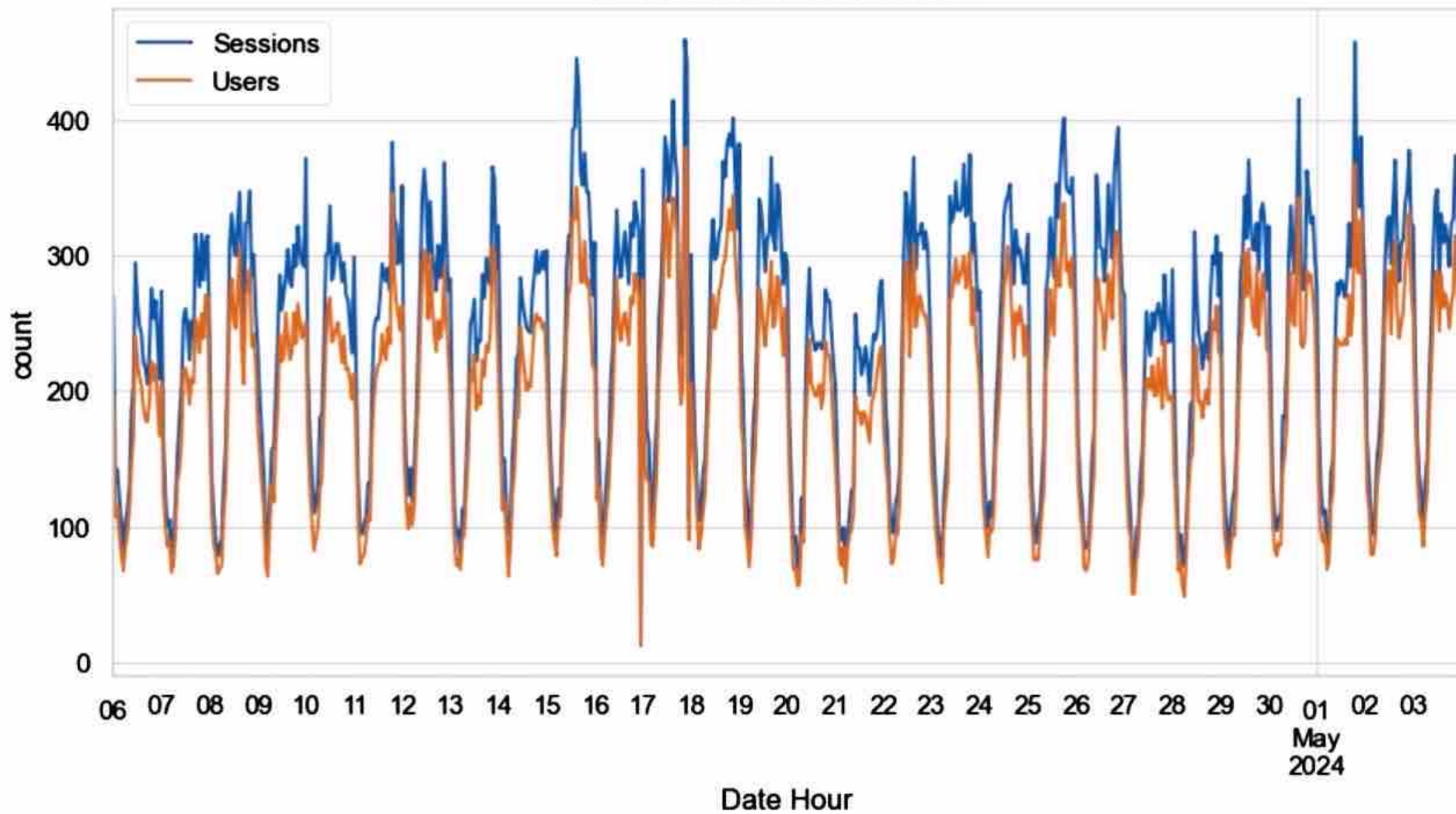
	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count	Hour
count	3177	3177.00000	3177.000000	3177.000000	3177.000000	3177.000000	3177.000000	3177.000000	3177.000000	3177.000000
mean	2024-04-20 01:22:32.974504	41.68870	50.871262	28.172490	66.686942	0.606409	4.676351	0.503413	240.871262	11.793201
min	2024-04-06 00:00:00	0.00000	1.000000	0.000000	0.000000	0.000000	1.000000	0.000000	1.000000	0.000000
25%	2024-04-13 02:00:00	20.00000	24.000000	13.000000	32.062500	0.561404	3.750000	0.442857	103.000000	6.000000
50%	2024-04-20 03:00:00	42.00000	51.000000	27.000000	49.115385	0.666667	4.410256	0.546392	225.000000	12.000000
75%	2024-04-26 22:00:00	60.00000	71.000000	41.000000	71.537037	0.750000	5.219780	0.633333	339.000000	18.000000
max	2024-05-03 23:00:00	160.00000	219.000000	125.000000	4525.000000	2.000000	56.000000	1.000000	1117.000000	23.000000
std	NaN	28.92827	36.031614	20.297391	127.295977	0.264227	2.797397	0.228385	181.079481	6.882770


```
[40]: sns.set(style='whitegrid')
```

Sessions and users over time

```
[43]: plt.figure(figsize=(10,5))
df.groupby("Date Hour")[["Sessions","Users"]].sum().plot(ax=plt.gca())
plt.title("Sessions and users over time")
plt.xlabel("Date Hour")
plt.ylabel("count")
plt.show()
```

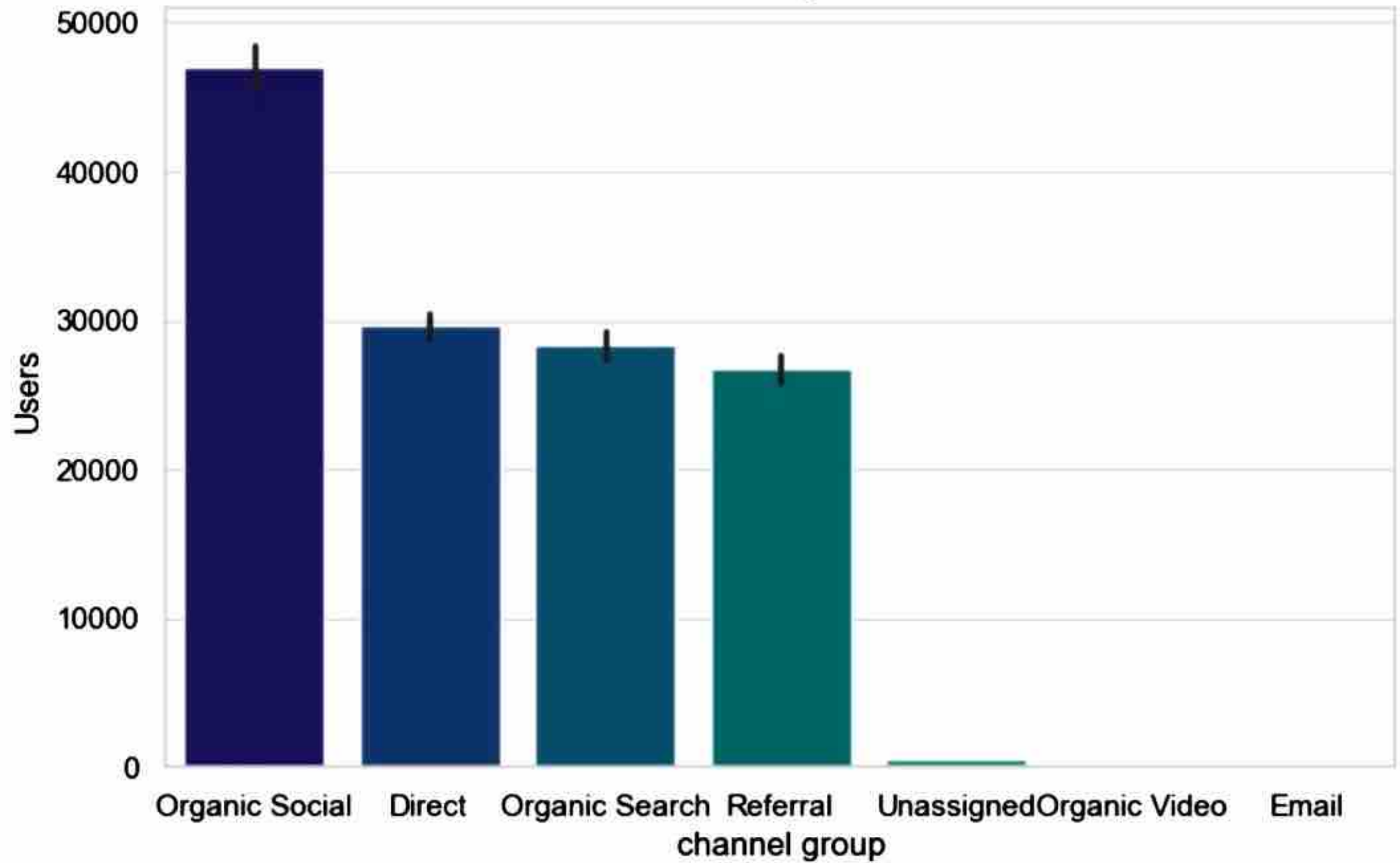
Sessions and users over time



total users by Channels

```
plt.figure(figsize=(8,5))
sns.barplot(data=df, x="channel group", y="Users", estimator = np.sum, palette = "viridis")
plt.title("Total users by channel")
plt.xticks(rotation=45)
plt.show()
```

Total users by channel



```
[50]: df.head()
```

```
[50]:
```

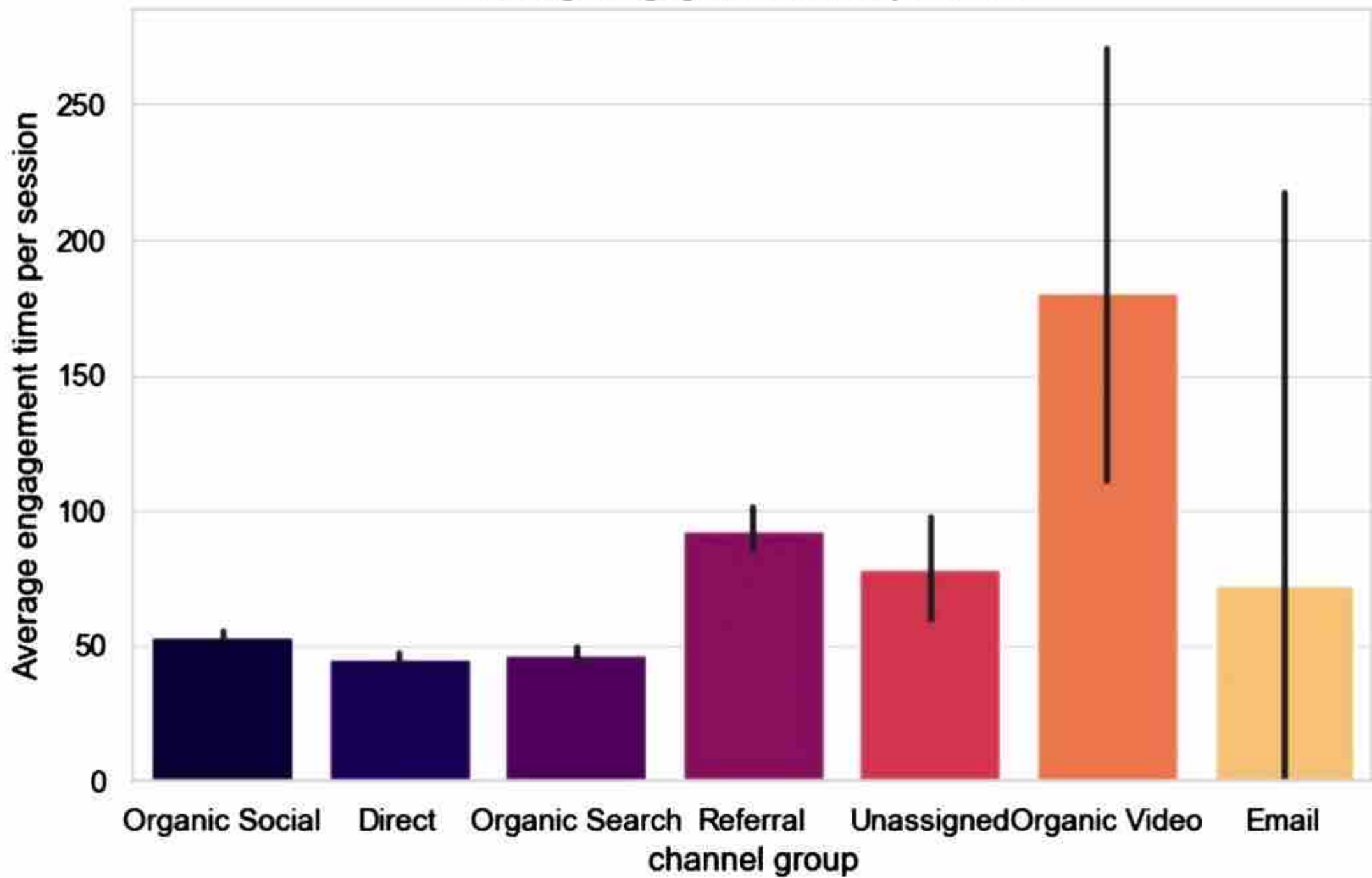
	channel group	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count	Hour
0	Organic Social	2024-04-17 21:00:00	160	206	103	59.315534	0.643750	4.694175	0.500000	967	21
1	Organic Social	2024-05-01 19:00:00	160	219	110	40.315068	0.687500	4.630137	0.502283	1014	19
2	Direct	2024-04-16 22:00:00	158	192	94	39.020833	0.594937	4.057292	0.489583	779	22
3	Organic Social	2024-04-13 21:00:00	152	178	96	30.488764	0.631579	5.056180	0.539326	900	21
4	Organic Social	2024-04-15 16:00:00	151	211	93	64.165877	0.615894	4.317536	0.440758	911	16

```
#average engagement time by channel
```

```
plt.figure(figsize=(8,5))
```

```
[52]: plt.figure(figsize=(8,5))
sns.barplot(data=df, x="channel group", y="Average engagement time per session", estimator = np.mean, palette = "magma")
plt.title("Average engagement time by channel")
plt.xticks(rotation=45)
plt.show()
```


Average engagement time by channel



```
[ ]: #AVG engagement Rate distribution by channel
```

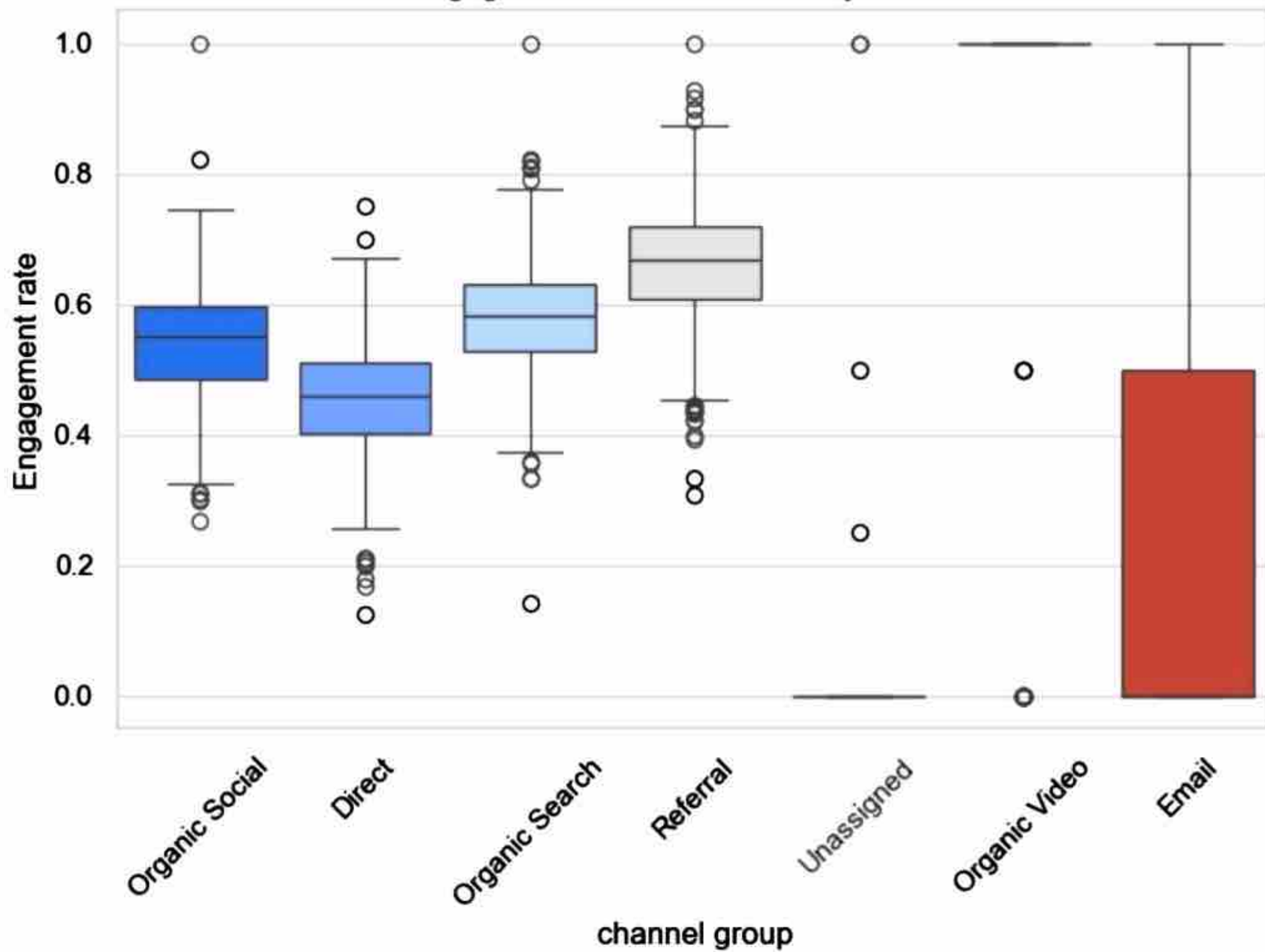
```
[56]: plt.figure(figsize=(8, 5))  
sns.boxplot(data=df, x="channel group", y="Engagement rate", palette = "coolwarm")  
plt.title("engagement Rate distribution by channel")  
plt.xticks(rotation=45)  
plt.show()
```

C:\Users\Srujan\AppData\Local\Temp\ipykernel_25500\318474389.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x="channel group", y="Engagement rate", palette = "coolwarm")
```

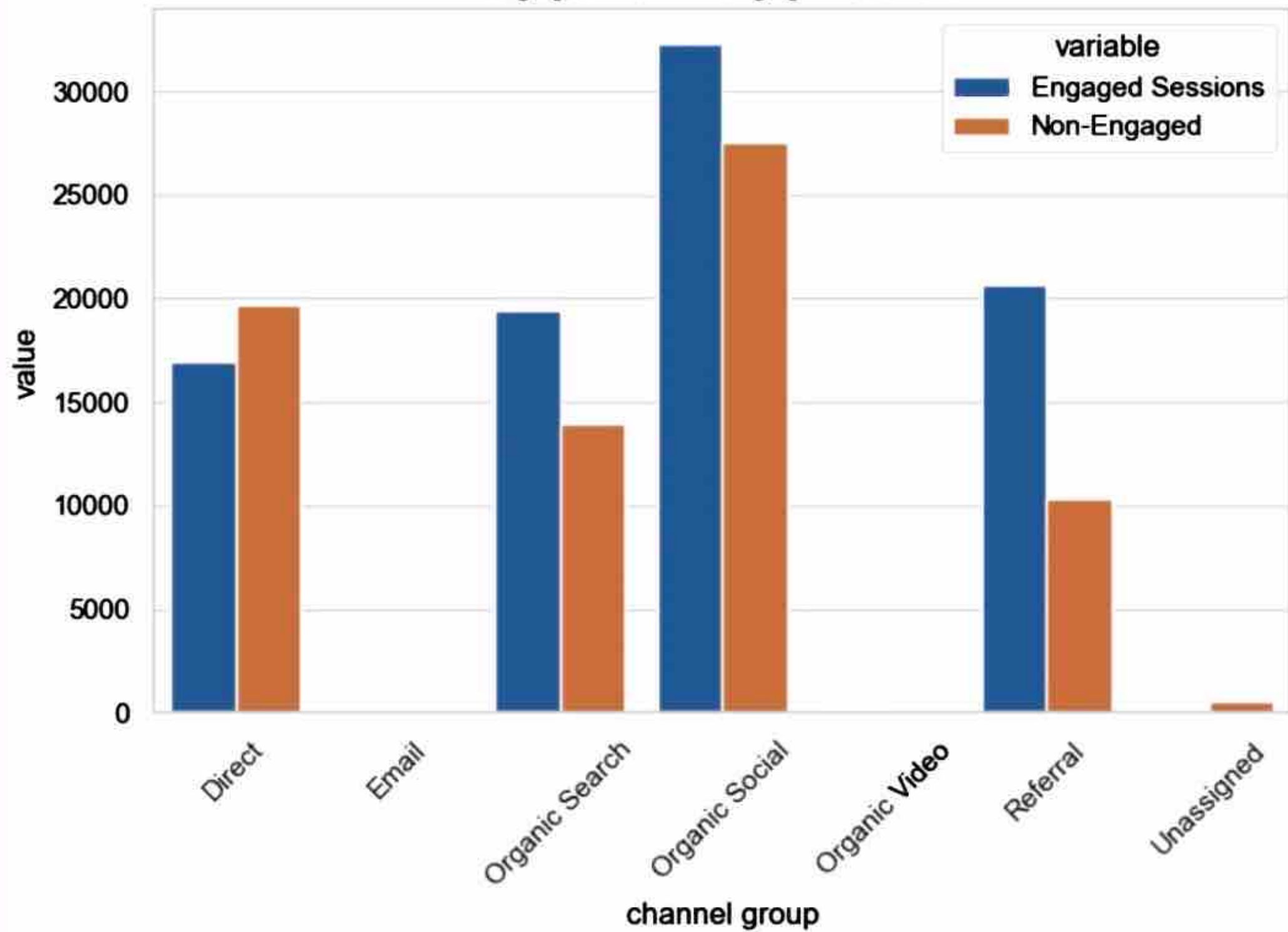
engagement Rate distribution by channel



```
[ ]: #Engaged VS non-engaged sessions
```

```
[59]: session_df = df.groupby("channel group")["Sessions", "Engaged Sessions"].sum().reset_index()  
      session_df["Non-Engaged"] = session_df["Sessions"] - session_df["Engaged Sessions"]  
      session_df_melted = session_df.melt(id_vars="channel group", value_vars=["Engaged Sessions", "Non-Engaged"])  
      plt.figure(figsize=(8, 5))  
      sns.barplot(data=session_df_melted, x="channel group", y="value", hue="variable")  
      plt.title("engaged VS non engaged sessions")  
      plt.xticks(rotation=45)  
      plt.show()
```

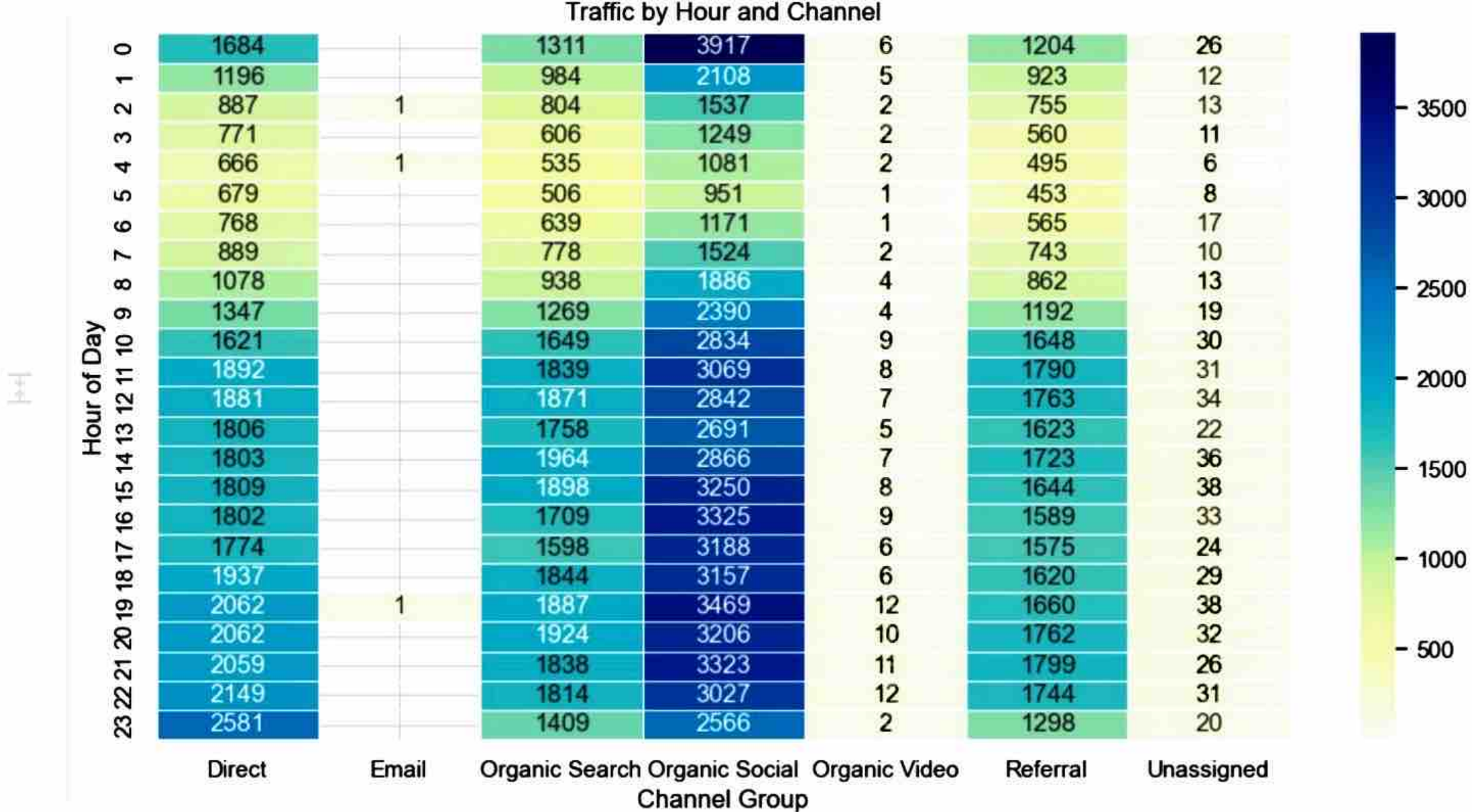
engaged VS non engaged sessions



```
[35]: heatmap_data = df.groupby(["Hour", "channel group"])["Sessions"].sum().unstack()
```

```
plt.figure(figsize=(12, 6))  
sns.heatmap(heatmap_data, cmap="YlGnBu", linewidths=.5, annot=True, fmt='.0f')  
plt.title("Traffic by Hour and Channel")  
plt.xlabel("Channel Group")  
plt.ylabel("Hour of Day")
```

```
[35]: Text(117.24999999999999, 0.5, 'Hour of Day')
```

[36]: df.head()

[36]:

	channel group	Date Hour	Users	Sessions	Engaged Sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Event count	Hour
0	Direct	2024-04-16 23:00:00	237	300	144	47.526667	0.607595	4.673333	0.480000	1402	23
1	Organic Social	2024-04-17 19:00:00	208	267	132	32.097378	0.634615	4.295880	0.494382	1147	19
2	Direct	2024-04-17 23:00:00	188	233	115	39.939914	0.611702	4.587983	0.493562	1069	23
3	Organic Social	2024-04-17 18:00:00	187	256	125	32.160156	0.668449	4.078125	0.488281	1044	18
4	Organic Social	2024-04-17 20:00:00	175	221	112	46.918552	0.640000	4.529412	0.506787	1001	20

Engagement Rate vs Sessions Over Time

